Derivatives and Inversion Theory for Plane-Wave Reflectivity

Kennett Recursion, Global Matrix Implicit Differentiation,
and Levenberg–Marquardt Inversion

Abstract

We present the complete derivative theory for plane-wave PP reflectivity inversion in stratified elastic media.

Part I (Sections 1–8) derives the Fréchet (first-order) derivatives via tangent-linear differentiation of the Kennett addition formula, equivalent to the analytical kernels of Dietrich & Kormendi (1990). The structure of the second-order (Hessian) derivatives is presented, identifying the cross-coupling terms that make same-layer Hessian elements combinatorially complex.

Part II (Section 9) derives the Jacobian and Hessian of the Global Matrix Method (Chin, Hedstrom & Thigpen, 1984) via implicit differentiation of the linear system $\mathbf{Gx} = \mathbf{b}$, showing that each derivative order reduces to a single back-substitution with the already-factored $\mathbf{G}$. The Block-Riccati sweep solver (Section 9.5) reduces the forward solve from $O(N^3)$ to $O(N)$ while preserving the implicit differentiation advantage through a chain of 4×4 `torch.linalg.solve` calls.

Part III (Section 10) presents the Levenberg–Marquardt inversion framework: the misfit functional, gradient, exact Hessian decomposition into Gauss–Newton and second-order residual terms, the damped Newton update, and the log-parameter transformation for positivity.

Convention: $\exp(-i\omega t)$ inverse Fourier transform; depth positive downward; $\mathrm{Im}(\eta) > 0$ for vertical slowness (radiation condition).

Contents

1 The Kennett Reflectivity **2**

2 Kennett Addition Formula **2**

3 Tangent-Linear Kennett Recursion **3**

4 Primitive Derivative Chains **3**
4.1 Complex slowness from velocity . . . 3
4.2 Vertical slowness . . . 3
4.3 P-wave velocity α_j . . . 4
4.4 S-wave velocity β_j . . . 4
4.5 Density ρ_j . . . 4
4.6 Thickness h_j . . . 4

5 Scattering Matrix Derivatives (Solid–Solid) **4**
5.1 Kennett intermediate variables . . . 4
5.2 Unnormalised scattering elements . . . 5
5.3 Tangent of the intermediate variables . . . 5
5.4 Tangent of the normalised variables . . . 5

5.5 Tangent of the scattering elements . . . 5
5.6 Tangent of the 2×2 scattering matrices . . . 6

6 Locality of the Perturbation **6**

7 Complete Jacobian **7**

8 Equivalence to Dietrich & Kormendi (1990) **7**
8.1 The four partitions . . . 7
8.2 Internal structure of P_3 . . . 8
8.3 Correspondence to the D&K Green's functions . . . 8

9 Ocean-Bottom Interface (Acoustic–Elastic) **8**

10 Second-Order Derivatives (Hessian) **9**
10.1 Structure of the misfit Hessian . . . 9
10.2 Same-layer cross-terms . . . 9
10.3 Magnitude of cross-terms . . . 9
10.4 Automatic differentiation resolves the problem . . . 9

11 Global Matrix Method: Implicit Differentiation **10**
11.1 Jacobian (first derivative) . . . 10
11.2 Hessian (second derivative) . . . 10
11.3 Cost comparison . . . 11
11.4 Block-tridiagonal structure and the Riccati connection . . . 11
11.5 Differentiable Block-Riccati Solver . . . 12

12 Levenberg–Marquardt Inversion **14**
12.1 Misfit functional . . . 14
12.2 Gradient . . . 14
12.3 Hessian decomposition . . . 14
12.4 The Levenberg–Marquardt update . . . 15
12.5 Log-parameter transformation . . . 15
12.6 Convergence properties . . . 15

1 The Kennett Reflectivity

The PP reflectivity of a stratified elastic half-space with N sub-ocean layers is

$$R(\omega, p) = e_{a,0}^2 \, \mathbf{RR}_d^{(0)}[0, 0], \tag{1}$$

where $e_{a,0}^2 = \exp(2i\omega\eta_0 h_0)$ is the two-way ocean phase and $\mathbf{RR}_d^{(0)}$ is the cumulative downgoing reflection matrix at the ocean bottom, computed by the upward recursion described below.

2 Kennett Addition Formula

At interface i (between layers i and $i{+}1$), the cumulative reflection from below is updated by

$$\mathbf{RR}_d^{(i)} = \mathbf{R}_d^{(i)} + \mathbf{T}_u^{(i)} \, \mathbf{M} \, \mathbf{U} \, \mathbf{T}_d^{(i)}, \tag{2}$$

where

$$\mathbf{M} = \mathbf{E}_{i+1} \, \mathbf{RR}_d^{(i+1)} \, \mathbf{E}_{i+1}, \qquad \mathbf{U} = \left(\mathbf{I} - \mathbf{R}_u^{(i)} \, \mathbf{M}\right)^{-1}, \tag{3}$$

and the phase matrix is

$$\mathbf{E}_j = \mathrm{diag}\big(e_{a,j},\ e_{b,j}\big), \qquad e_{a,j} = \exp(i\omega\eta_j h_j), \quad e_{b,j} = \exp(i\omega\nu_j h_j). \tag{4}$$

Here η_j and ν_j are the P- and S-wave vertical slownesses, h_j is the layer thickness, and $\mathbf{R}_d^{(i)}$, $\mathbf{R}_u^{(i)}$, $\mathbf{T}_u^{(i)}$, $\mathbf{T}_d^{(i)}$ are the 2×2 scattering matrices at interface i.

Initialisation: $\mathbf{RR}_d^{(N-1)} = \mathbf{0}$ (radiation condition at the half-space).

3 Tangent-Linear Kennett Recursion

Differentiating (2) with respect to an arbitrary model parameter m_j yields the tangent-linear recursion for $\delta\mathbf{RR}_d$:

$$\boxed{\delta\mathbf{RR}_d^{(i)} = \delta\mathbf{R}_d + \delta\mathbf{T}_u\,\mathbf{Z}\,\mathbf{T}_d + \mathbf{T}_u\,\delta\mathbf{Z}\,\mathbf{T}_d + \mathbf{T}_u\,\mathbf{Z}\,\delta\mathbf{T}_d} \tag{5}$$

where $\mathbf{Z} = \mathbf{M}\,\mathbf{U}$ and the subsidiary tangents are:

$$\delta\mathbf{Z} = \delta\mathbf{M}\,\mathbf{U} + \mathbf{M}\,\delta\mathbf{U}, \tag{6}$$

$$\delta\mathbf{U} = \mathbf{U}\big(\delta\mathbf{R}_u\,\mathbf{M} + \mathbf{R}_u\,\delta\mathbf{M}\big)\mathbf{U}, \tag{7}$$

$$\delta\mathbf{M} = \delta\mathbf{E}\,\mathbf{RR}_d\,\mathbf{E} + \mathbf{E}\,\delta\mathbf{RR}_d\,\mathbf{E} + \mathbf{E}\,\mathbf{RR}_d\,\delta\mathbf{E}. \tag{8}$$

Remark 1. Equation (7) follows from the identity $\mathrm{d}(\mathbf{W}^{-1}) = \mathbf{W}^{-1}\,\mathrm{d}\mathbf{W}\,\mathbf{W}^{-1}$ applied to $\mathbf{W} = \mathbf{I} - \mathbf{R}_u\,\mathbf{M}$.

Initialisation: $\delta\mathbf{RR}_d^{(N-1)} = \mathbf{0}$.

4 Primitive Derivative Chains

For a perturbation of parameter m_j in layer j, the following chains provide the tangent inputs to the scattering matrices and phase factors.

4.1 Complex slowness from velocity

From the Futterman attenuation model:

$$s = \frac{2Q^2 + 2Qi}{(1 + 4Q^2)\,v}, \tag{9}$$

where Q is the quality factor (held fixed). Since s is proportional to $1/v$,

$$\boxed{\frac{\partial s}{\partial v} = -\frac{s}{v}.} \tag{10}$$

4.2 Vertical slowness

With $\eta = \sqrt{s^2 - p^2}$ and the branch chosen so that $\mathrm{Im}(\eta) > 0$:

$$\frac{\partial \eta}{\partial s} = \frac{s}{\eta}. \tag{11}$$

This holds for either sign of η: the branch flip $\eta \to -\eta$ leaves s/η invariant.

4.3 P-wave velocity α_j

$$\frac{\partial s_{p,j}}{\partial \alpha_j} = -\frac{s_{p,j}}{\alpha_j}, \tag{12}$$

$$\frac{\partial \eta_j}{\partial \alpha_j} = \frac{s_{p,j}}{\eta_j} \cdot \frac{\partial s_{p,j}}{\partial \alpha_j} = -\frac{s_{p,j}^2}{\eta_j\, \alpha_j}, \tag{13}$$

$$\frac{\partial e_{a,j}}{\partial \alpha_j} = i\omega h_j \, \frac{\partial \eta_j}{\partial \alpha_j} \, e_{a,j}. \tag{14}$$

4.4 S-wave velocity β_j

$$\frac{\partial s_{s,j}}{\partial \beta_j} = -\frac{s_{s,j}}{\beta_j}, \qquad \frac{\partial \nu_j}{\partial \beta_j} = -\frac{s_{s,j}^2}{\nu_j\, \beta_j}, \qquad \frac{\partial \tilde{\beta}_j}{\partial \beta_j} = \frac{\tilde{\beta}_j}{\beta_j}, \tag{15}$$

$$\frac{\partial e_{b,j}}{\partial \beta_j} = i\omega h_j \, \frac{\partial \nu_j}{\partial \beta_j} \, e_{b,j}, \tag{16}$$

where $\tilde{\beta}_j = 1/s_{s,j}$ is the complex S-velocity entering the scattering matrices.

4.5 Density ρ_j

Density enters the scattering matrices directly. It does not affect the phase factors ($\delta e_{a,j} = \delta e_{b,j} = 0$).

4.6 Thickness h_j

$$\frac{\partial e_{a,j}}{\partial h_j} = i\omega\eta_j \, e_{a,j}, \qquad \frac{\partial e_{b,j}}{\partial h_j} = i\omega\nu_j \, e_{b,j}. \tag{17}$$

Thickness does not affect the scattering matrices.

5 Scattering Matrix Derivatives (Solid–Solid)

5.1 Kennett intermediate variables

At interface i between layers with properties $(\eta_1, \nu_1, \rho_1, \tilde{\beta}_1)$ above and $(\eta_2, \nu_2, \rho_2, \tilde{\beta}_2)$ below, define:

$$\mu_k = \rho_k \, \tilde{\beta}_k^2, \qquad \Delta\mu = \mu_2 - \mu_1, \qquad d = 2\Delta\mu, \tag{18}$$

$$a = \Delta\rho - p^2 d, \qquad b = \rho_2 - p^2 d, \qquad c = \rho_1 + p^2 d, \tag{19}$$

$$E = b\eta_1 + c\eta_2, \qquad F = b\nu_1 + c\nu_2, \tag{20}$$

$$G = a - d\eta_1\nu_2, \qquad H = a - d\eta_2\nu_1, \tag{21}$$

$$D = EF + GHp^2. \tag{22}$$

5.2 Unnormalised scattering elements

Define normalised variables $\hat{E} = E/D$, etc. The six independent scattering elements are:

$$Q = (b\eta_1 - c\eta_2)\,\hat{F}, \qquad R = (a + d\eta_1\nu_2)\,\hat{H}\,p^2, \tag{23}$$

$$S = \frac{(ab + cd\eta_2\nu_2)\,p}{D}, \qquad T = (b\nu_1 - c\nu_2)\,\hat{E}, \tag{24}$$

$$U = (a + d\eta_2\nu_1)\,\hat{G}\,p^2, \qquad V = \frac{(ac + bd\eta_1\nu_1)\,p}{D}. \tag{25}$$

5.3 Tangent of the intermediate variables

For a perturbation $(\delta\eta_1, \delta\nu_1, \delta\rho_1, \delta\tilde{\beta}_1, \delta\eta_2, \delta\nu_2, \delta\rho_2, \delta\tilde{\beta}_2)$:

$$\delta\mu_k = \delta\rho_k\,\tilde{\beta}_k^2 + 2\rho_k\tilde{\beta}_k\,\delta\tilde{\beta}_k, \tag{26}$$

$$\delta d = 2(\delta\mu_2 - \delta\mu_1), \tag{27}$$

$$\delta a = \delta(\Delta\rho) - p^2\,\delta d, \tag{28}$$

$$\delta b = \delta\rho_2 - p^2\,\delta d, \tag{29}$$

$$\delta c = \delta\rho_1 + p^2\,\delta d, \tag{30}$$

$$\delta E = \delta b\,\eta_1 + b\,\delta\eta_1 + \delta c\,\eta_2 + c\,\delta\eta_2, \tag{31}$$

$$\delta F = \delta b\,\nu_1 + b\,\delta\nu_1 + \delta c\,\nu_2 + c\,\delta\nu_2, \tag{32}$$

$$\delta G = \delta a - \delta d\,\eta_1\nu_2 - d\,\delta\eta_1\,\nu_2 - d\,\eta_1\,\delta\nu_2, \tag{33}$$

$$\delta H = \delta a - \delta d\,\eta_2\nu_1 - d\,\delta\eta_2\,\nu_1 - d\,\eta_2\,\delta\nu_1, \tag{34}$$

$$\delta D = \delta E\,F + E\,\delta F + (\delta G\,H + G\,\delta H)\,p^2. \tag{35}$$

5.4 Tangent of the normalised variables

Using the quotient rule $\delta(X/D) = (\delta X \cdot D - X \cdot \delta D)/D^2$:

$$\delta\hat{E} = \frac{\delta E \cdot D - E \cdot \delta D}{D^2}, \quad \delta\hat{F} = \frac{\delta F \cdot D - F \cdot \delta D}{D^2}, \quad \text{etc.} \tag{36}$$

5.5 Tangent of the scattering elements

$$\delta Q = \delta(b\eta_1 - c\eta_2)\,\hat{F} + (b\eta_1 - c\eta_2)\,\delta\hat{F}, \tag{37}$$

$$\delta R = \delta(a + d\eta_1\nu_2)\,\hat{H}\,p^2 + (a + d\eta_1\nu_2)\,\delta\hat{H}\,p^2, \tag{38}$$

$$\delta S = \frac{\delta(ab + cd\eta_2\nu_2) \cdot D - (ab + cd\eta_2\nu_2) \cdot \delta D}{D^2}\,p, \tag{39}$$

$$\delta T = \delta(b\nu_1 - c\nu_2)\,\hat{E} + (b\nu_1 - c\nu_2)\,\delta\hat{E}, \tag{40}$$

$$\delta U = \delta(a + d\eta_2\nu_1)\,\hat{G}\,p^2 + (a + d\eta_2\nu_1)\,\delta\hat{G}\,p^2, \tag{41}$$

$$\delta V = \frac{\delta(ac + bd\eta_1\nu_1) \cdot D - (ac + bd\eta_1\nu_1) \cdot \delta D}{D^2}\,p. \tag{42}$$

The product-rule expansions needed in (37)–(42) are:

$$\delta(b\eta_1 - c\eta_2) = \delta b\,\eta_1 + b\,\delta\eta_1 - \delta c\,\eta_2 - c\,\delta\eta_2, \tag{43}$$
$$\delta(a + d\eta_1\nu_2) = \delta a + \delta d\,\eta_1\nu_2 + d\,\delta\eta_1\,\nu_2 + d\,\eta_1\,\delta\nu_2, \tag{44}$$
$$\delta(b\nu_1 - c\nu_2) = \delta b\,\nu_1 + b\,\delta\nu_1 - \delta c\,\nu_2 - c\,\delta\nu_2, \tag{45}$$
$$\delta(a + d\eta_2\nu_1) = \delta a + \delta d\,\eta_2\nu_1 + d\,\delta\eta_2\,\nu_1 + d\,\eta_2\,\delta\nu_1, \tag{46}$$
$$\delta(ab + cd\eta_2\nu_2) = \delta a\,b + a\,\delta b + \delta c\,d\eta_2\nu_2 + c\,\delta d\,\eta_2\nu_2 + cd\,\delta\eta_2\,\nu_2 + cd\,\eta_2\,\delta\nu_2, \tag{47}$$
$$\delta(ac + bd\eta_1\nu_1) = \delta a\,c + a\,\delta c + \delta b\,d\eta_1\nu_1 + b\,\delta d\,\eta_1\nu_1 + bd\,\delta\eta_1\,\nu_1 + bd\,\eta_1\,\delta\nu_1. \tag{48}$$

5.6 Tangent of the 2×2 scattering matrices

Define the auxiliary square-root quantities:

$$\hat{\eta}_k = \sqrt{\eta_k}, \quad \hat{\nu}_k = \sqrt{\nu_k}, \quad \hat{\rho}_k = \sqrt{\rho_k}, \quad \hat{z}_{a,k} = \hat{\eta}_k\hat{\rho}_k, \quad \hat{z}_{b,k} = \hat{\nu}_k\hat{\rho}_k. \tag{49}$$

Their tangents are $\delta\hat{\eta}_k = \delta\eta_k/(2\hat{\eta}_k)$, $\delta\hat{\rho}_k = \delta\rho_k/(2\hat{\rho}_k)$, etc., and

$$\delta\hat{z}_{a,k} = \delta\hat{\eta}_k\,\hat{\rho}_k + \hat{\eta}_k\,\delta\hat{\rho}_k, \qquad \delta\hat{z}_{b,k} = \delta\hat{\nu}_k\,\hat{\rho}_k + \hat{\nu}_k\,\delta\hat{\rho}_k. \tag{50}$$

The scattering matrices and their tangents are:

$$\delta\mathbf{R}_d = \begin{pmatrix} \delta Q - \delta R & -2i\big(\delta\hat{\eta}_1\,\hat{\nu}_1 + \hat{\eta}_1\,\delta\hat{\nu}_1\big)S - 2i\hat{\eta}_1\hat{\nu}_1\,\delta S \\ \text{(sym)} & \delta T - \delta U \end{pmatrix}, \tag{51}$$

$$\delta\mathbf{T}_d = \begin{pmatrix} 2\big(\delta\hat{z}_{a,1}\hat{z}_{a,2} + \hat{z}_{a,1}\delta\hat{z}_{a,2}\big)\hat{F} + 2\hat{z}_{a,1}\hat{z}_{a,2}\,\delta\hat{F} & \cdots \\ \cdots & 2\big(\delta\hat{z}_{b,1}\hat{z}_{b,2} + \hat{z}_{b,1}\delta\hat{z}_{b,2}\big)\hat{E} + 2\hat{z}_{b,1}\hat{z}_{b,2}\,\delta\hat{E} \end{pmatrix}, \tag{52}$$

$$\delta\mathbf{T}_u = (\delta\mathbf{T}_d)^T \qquad \text{(reciprocity)}, \tag{53}$$

$$\delta\mathbf{R}_u = \begin{pmatrix} -(\delta Q + \delta U) & -2i\big(\delta\hat{\eta}_2\hat{\nu}_2 + \hat{\eta}_2\delta\hat{\nu}_2\big)V - 2i\hat{\eta}_2\hat{\nu}_2\,\delta V \\ \text{(sym)} & -(\delta T + \delta R) \end{pmatrix}. \tag{54}$$

6 Locality of the Perturbation

For a parameter m_j belonging to layer j, non-zero tangent inputs occur only at:

Source	**Affected interface**	**Role of layer j**
$\delta\mathbf{R}_d$, $\delta\mathbf{R}_u$, $\delta\mathbf{T}_u$, $\delta\mathbf{T}_d$	$j-1$	"layer 2" (below)
$\delta\mathbf{R}_d$, $\delta\mathbf{R}_u$, $\delta\mathbf{T}_u$, $\delta\mathbf{T}_d$	j (if $< N-1$)	"layer 1" (above)
$\delta\mathbf{E}_j$	phase through layer j	$\delta e_{a,j}$, $\delta e_{b,j}$

All other interfaces have $\delta\mathbf{R}_d = \delta\mathbf{R}_u = \delta\mathbf{T}_u = \delta\mathbf{T}_d = \mathbf{0}$ and $\delta\mathbf{E} = \mathbf{0}$. The tangent-linear recursion (5) merely propagates $\delta\mathbf{R}\mathbf{R}_d$ through the unperturbed Kennett addition formula at those interfaces.

7 Complete Jacobian

The full Jacobian has shape (nfreq) $\times$ $(4N-1)$ where N is the number of sub-ocean layers:

$$J_{ij} = \frac{\partial R(\omega_i)}{\partial m_j} = e_{a,0}^2(\omega_i) \cdot \delta \mathbf{RR}_d^{(0)}[0,0]\Big|_{m_j}. \tag{55}$$

The parameter vector is:

$$\mathbf{m} = \big[\alpha_1, \ldots, \alpha_N,\ \beta_1, \ldots, \beta_N,\ \rho_1, \ldots, \rho_N,\ h_1, \ldots, h_{N-1}\big]. \tag{56}$$

Remark 2. The half-space layer ($h_N = \infty$) is excluded from the parameter vector because its thickness is not a free parameter.

8 Equivalence to Dietrich & Kormendi (1990)

The tangent-linear recursion derived above is the *forward-mode automatic differentiation* of the Kennett recursion. It computes the exact same quantity as the Dietrich & Kormendi (1990) analytical Fréchet derivative, expressed as a recursion over interfaces rather than an integral over Green's functions.

D&K's Fréchet kernel for a continuously varying medium is

$$\delta R = \int_0^\infty G^{\uparrow}(z)\ \delta\mathbf{P}(z)\ G^{\downarrow}(z)\,\mathrm{d}z, \tag{57}$$

where $G^{\uparrow}$ and $G^{\downarrow}$ are the upgoing and downgoing Green's function operators and $\delta\mathbf{P}$ is the first-order perturbation of the propagator. For a stack of homogeneous layers this integral collapses to a finite sum over the interfaces where elastic properties are discontinuous.

8.1 The four partitions

At each interface i where scattering is perturbed, the tangent-linear recursion (5) decomposes the contribution to $\delta\mathbf{RR}_d^{(i)}$ into exactly four partitions:

$$\delta\mathbf{RR}_d^{(i)} = \underbrace{\delta\mathbf{R}_d^{(i)}}_{P_1} + \underbrace{\delta\mathbf{T}_u^{(i)}\,\mathbf{Z}^{(i)}\,\mathbf{T}_d^{(i)}}_{P_2} + \underbrace{\mathbf{T}_u^{(i)}\,\delta\mathbf{Z}^{(i)}\,\mathbf{T}_d^{(i)}}_{P_3} + \underbrace{\mathbf{T}_u^{(i)}\,\mathbf{Z}^{(i)}\,\delta\mathbf{T}_d^{(i)}}_{P_4}, \tag{58}$$

where $\mathbf{Z}^{(i)} = \mathbf{M}^{(i)}\,\mathbf{U}^{(i)}$ as before. Each partition corresponds to a distinct physical scattering pathway:

	Partition	**Physical pathway**
P_1	$\delta\mathbf{R}_d$	Downgoing wave reflects off the *perturbed* interface. No interaction with the stack below.
P_2	$\delta\mathbf{T}_u\,\mathbf{Z}\,\mathbf{T}_d$	Wave transmits down (unperturbed), reverberates in the stack below (unperturbed), then transmits up through the *perturbed* interface.
P_3	$\mathbf{T}_u\,\delta\mathbf{Z}\,\mathbf{T}_d$	Wave enters and exits the interface unperturbed, but the *reverberation* between $\mathbf{R}_u$ and the stack below is perturbed (contains $\delta\mathbf{R}_u$, $\delta\mathbf{E}$, and propagated $\delta\mathbf{RR}_d$; see below).
P_4	$\mathbf{T}_u\,\mathbf{Z}\,\delta\mathbf{T}_d$	Wave transmits down through the *perturbed* interface, reverberates unperturbed below, and transmits up unperturbed.

8.2 Internal structure of P_3

Partition P_3 is the richest because $\delta\mathbf{Z} = \delta\mathbf{M}\,\mathbf{U} + \mathbf{M}\,\delta\mathbf{U}$ contains three physically distinct sub-contributions via $\delta\mathbf{M}$ and $\delta\mathbf{U}$:

(a) **Phase perturbation** ($\delta\mathbf{E}$ terms in $\delta\mathbf{M}$): $\delta\mathbf{E}\,\mathbf{R}\mathbf{R}_d\,\mathbf{E} + \mathbf{E}\,\mathbf{R}\mathbf{R}_d\,\delta\mathbf{E}$. A wave traversing layer j accumulates a perturbed phase; the cumulative reflection from below is unperturbed.

(b) **Propagated tangent from below** ($\delta\mathbf{R}\mathbf{R}_d$ term in $\delta\mathbf{M}$): $\mathbf{E}\,\delta\mathbf{R}\mathbf{R}_d\,\mathbf{E}$. First-order perturbations originating at deeper interfaces propagate upward through the unperturbed phase and enter the current interface.

(c) **Upgoing reflection perturbation** ($\delta\mathbf{R}_u$ in $\delta\mathbf{U}$): $\mathbf{U}\,\delta\mathbf{R}_u\,\mathbf{M}\,\mathbf{U}$. The reverberative coupling between the interface and the stack below is perturbed because $\mathbf{R}_u$ changes.

Combining these with the definition $\delta\mathbf{U} = \mathbf{U}(\delta\mathbf{R}_u\,\mathbf{M} + \mathbf{R}_u\,\delta\mathbf{M})\mathbf{U}$ from (7), partition P_3 expands to:

$$P_3 = \mathbf{T}_u\Big[\underbrace{\delta\mathbf{M}\,\mathbf{U}}_{\text{(a)+(b)}} + \underbrace{\mathbf{M}\,\mathbf{U}\big(\delta\mathbf{R}_u\,\mathbf{M} + \mathbf{R}_u\,\delta\mathbf{M}\big)\mathbf{U}}_{\text{(c) + reverb of (a)+(b)}}\Big]\mathbf{T}_d. \tag{59}$$

8.3 Correspondence to the D&K Green's functions

In (57), the operators $G^{\uparrow}$ and $G^{\downarrow}$ encode cumulative transmission through all overlying and underlying interfaces. In the Kennett recursive framework these operators are not formed explicitly; instead, the upward sweep of the tangent-linear recursion (5) through *unperturbed* interfaces acts as the propagator:

D&K operator	**Kennett equivalent**
$G^{\downarrow}(z_i)$	Encoded in $\mathbf{R}\mathbf{R}_d^{(i+1)}$: the cumulative downgoing reflection of the entire stack below interface i.
$G^{\uparrow}(z_i)$	Encoded in the unperturbed Kennett addition steps at interfaces $i-1, i-2, \ldots, 0$ that propagate $\delta\mathbf{R}\mathbf{R}_d$ from interface i up to the surface.
$\delta\mathbf{P}(z_i)$	The 4-partition decomposition: $(\delta\mathbf{R}_d, \delta\mathbf{T}_u, \delta\mathbf{R}_u, \delta\mathbf{T}_d)$ at the interface, plus $\delta\mathbf{E}$ in the adjacent layer. These are the "source terms" that inject the perturbation into the recursion.

The equivalence is exact: the tangent-linear recursion computes the same Fréchet derivative as D&K's integral kernel, partition by partition, with no approximation.

Proposition 1 (Numerical validation)**.** *The AD Jacobian (`torch.autograd`) agrees with this tangent-linear Jacobian (NumPy) to* $< 10^{-10}$ *relative error, and both agree with central finite differences to* $< 10^{-5}$.

9 Ocean-Bottom Interface (Acoustic–Elastic)

The ocean-bottom interface has the same structure as solid–solid (Section 5), with the simplifications:

- $\nu_1 = 0$ (no shear in ocean),
- $\tilde{\beta}_1 = 0$,
- $F = b$ (not $b\nu_1 + c\nu_2$),
- $H = -d\eta_2$ (not $a - d\eta_2\nu_1$),
- $\mathbf{R}_d$ has only the $[0, 0]$ entry nonzero (P→P only from above),

- $\mathbf{T}_d$ has column 1 zero (no downgoing S from ocean),
- $\mathbf{T}_u$ has row 1 zero (no upgoing S to ocean).

Ocean parameters (layer 0) are fixed: $\delta\eta_1 = \delta\rho_1 = 0$.

10 Second-Order Derivatives (Hessian)

10.1 Structure of the misfit Hessian

For a scalar L2 misfit $\chi = \sum_i |R(\omega_i) - R_{\text{obs}}(\omega_i)|^2$, the Hessian is:

$$H_{kl} = \frac{\partial^2 \chi}{\partial m_k \, \partial m_l} = \underbrace{\mathrm{Re}\sum_i \overline{\frac{\partial R_i}{\partial m_k}} \, \frac{\partial R_i}{\partial m_l}}_{\text{Gauss–Newton}} + \underbrace{\mathrm{Re}\sum_i \overline{r_i} \, \frac{\partial^2 R_i}{\partial m_k \, \partial m_l}}_{\text{second-order}}, \tag{60}$$

where $r_i = R(\omega_i) - R_{\text{obs}}(\omega_i)$.

The Gauss–Newton term uses only the first-order Jacobian. The second-order term involves $\partial^2 R / \partial m_k \, \partial m_l$, which requires differentiating the tangent-linear recursion a second time.

10.2 Same-layer cross-terms

When m_k and m_l belong to the *same* layer j, the second-order tangent-linear $\delta^2 \mathbf{RR}_d$ receives contributions from three sources:

S1: Scattering at interface $j-1$ (layer j below),
S2: Scattering at interface j (layer j above),
P: Phase through layer j.

The **cross-coupling terms** S1×S2, S1×P, and S2×P arise because the Kennett recursion chains these three contributions together. The key cross-term is in $\delta^2 \mathbf{M}$:

$$\delta^2 \mathbf{M} = 2\,\delta_k \mathbf{E}\;\delta_l \mathbf{RR}_d\;\mathbf{E} + 2\,\mathbf{E}\;\delta_l \mathbf{RR}_d\;\delta_k \mathbf{E} + \delta_k \mathbf{E}\;\mathbf{RR}_d\;\delta_l \mathbf{E} + \delta_l \mathbf{E}\;\mathbf{RR}_d\;\delta_k \mathbf{E} + \mathbf{E}\;\delta^2 \mathbf{RR}_d\;\mathbf{E} + \cdots \tag{61}$$

where δ_k and δ_l denote tangent-linear operations with respect to m_k and m_l respectively.

The terms $\delta_k \mathbf{E} \cdot \delta_l \mathbf{RR}_d \cdot \mathbf{E}$ couple the accumulated first-order tangent from below ($\delta_l \mathbf{RR}_d$) with the local phase perturbation ($\delta_k \mathbf{E}$). This is the scattering×phase cross-term that was identified as missing from the 1991 thesis derivation.

10.3 Magnitude of cross-terms

Numerical experiments on a 5-layer ocean–crust model (5% perturbation in α_2) show:

Cross-term	Relative to max diagonal
$\alpha \times h$	67%
$\alpha \times \rho$	17%
$\beta \times h$	45%

These cross-terms are large enough that the Gauss–Newton approximation (which neglects the second-order term entirely) and hand-derivations that miss cross-couplings are both inadequate for Newton-type inversion.

10.4 Automatic differentiation resolves the problem

The PyTorch implementation computes the exact Hessian via `torch.autograd.functional.hessian`, which differentiates the computational graph twice without any manual derivation. This automatically captures *all* cross-terms, resolving the combinatorial complexity that prevented completion of the 1991 hand derivation.

11 Global Matrix Method: Implicit Differentiation

The Global Matrix Method (Chin, Hedstrom & Thigpen, 1984) assembles all interface continuity conditions into a single linear system:

$$\mathbf{G}(\boldsymbol{\theta})\,\mathbf{x} = \mathbf{b}(\boldsymbol{\theta}), \tag{62}$$

where $\mathbf{G}$ is the global matrix encoding the displacement-stress continuity across all interfaces, $\mathbf{b}$ is the source vector, and $\mathbf{x}$ contains the displacement-stress coefficients at every interface. Both $\mathbf{G}$ and $\mathbf{b}$ depend on the model parameter vector $\boldsymbol{\theta} = (\alpha_1, \ldots, \alpha_N,\ \beta_1, \ldots, \beta_N,\ \rho_1, \ldots, \rho_N,\ h_1, \ldots, h_{N-1})$.

The reflectivity is extracted from $\mathbf{x}$ by a linear projection. Since $\mathbf{G}$ and $\mathbf{b}$ are assembled analytically from the layer $\mathbf{E}$-matrices, all derivatives of $\mathbf{G}$ and $\mathbf{b}$ with respect to $\boldsymbol{\theta}$ are available in closed form. The key advantage is that derivatives of the *solution* $\mathbf{x}$ are obtained by implicit differentiation of the linear system, without differentiating through the internal steps of the solver.

11.1 Jacobian (first derivative)

Differentiate (62) with respect to a model parameter θ_j:

$$\frac{\partial \mathbf{G}}{\partial \theta_j}\,\mathbf{x} + \mathbf{G}\,\frac{\partial \mathbf{x}}{\partial \theta_j} = \frac{\partial \mathbf{b}}{\partial \theta_j}. \tag{63}$$

Rearranging:

$$\boxed{\mathbf{G}\,\frac{\partial \mathbf{x}}{\partial \theta_j} = \frac{\partial \mathbf{b}}{\partial \theta_j} - \frac{\partial \mathbf{G}}{\partial \theta_j}\,\mathbf{x}.} \tag{64}$$

This is a linear system with the *same coefficient matrix* $\mathbf{G}$ as the forward problem. Since $\mathbf{G}$ has already been LU-factored during the forward solve, each Jacobian column costs only a single back-substitution — not a new factorisation.

11.2 Hessian (second derivative)

Differentiate (63) again with respect to a second parameter θ_k:

$$\frac{\partial^2 \mathbf{G}}{\partial \theta_j\,\partial \theta_k}\,\mathbf{x} + \frac{\partial \mathbf{G}}{\partial \theta_j}\,\frac{\partial \mathbf{x}}{\partial \theta_k} + \frac{\partial \mathbf{G}}{\partial \theta_k}\,\frac{\partial \mathbf{x}}{\partial \theta_j} + \mathbf{G}\,\frac{\partial^2 \mathbf{x}}{\partial \theta_j\,\partial \theta_k} = \frac{\partial^2 \mathbf{b}}{\partial \theta_j\,\partial \theta_k}. \tag{65}$$

Rearranging:

$$\boxed{\mathbf{G}\,\frac{\partial^2 \mathbf{x}}{\partial \theta_j\,\partial \theta_k} = \frac{\partial^2 \mathbf{b}}{\partial \theta_j\,\partial \theta_k} - \frac{\partial^2 \mathbf{G}}{\partial \theta_j\,\partial \theta_k}\,\mathbf{x} - \frac{\partial \mathbf{G}}{\partial \theta_j}\,\frac{\partial \mathbf{x}}{\partial \theta_k} - \frac{\partial \mathbf{G}}{\partial \theta_k}\,\frac{\partial \mathbf{x}}{\partial \theta_j}.} \tag{66}$$

Again the same matrix $\mathbf{G}$ appears on the left-hand side. The right-hand side requires only quantities already computed:

- the forward solution $\mathbf{x}$,
- the first derivatives $\partial \mathbf{x}/\partial \theta_j$ and $\partial \mathbf{x}/\partial \theta_k$ from the Jacobian step,
- the first and second derivatives of $\mathbf{G}$ and $\mathbf{b}$ with respect to $\boldsymbol{\theta}$, assembled analytically from the layer matrices.

For the diagonal Hessian element ($\theta_k = \theta_j$) equation (66) simplifies to:

$$\mathbf{G}\,\frac{\partial^2 \mathbf{x}}{\partial \theta_j^2} = \frac{\partial^2 \mathbf{b}}{\partial \theta_j^2} - \frac{\partial^2 \mathbf{G}}{\partial \theta_j^2}\,\mathbf{x} - 2\,\frac{\partial \mathbf{G}}{\partial \theta_j}\,\frac{\partial \mathbf{x}}{\partial \theta_j}. \tag{67}$$

11.3 Cost comparison

Quantity	Kennett recursion	Global Matrix
Forward $\mathbf{x}$	Recursive sweep: $O(N)$ 2×2 ops	LU factorise $\mathbf{G}$: $O(N^3)$
Jacobian $\partial\mathbf{x}/\partial\theta_j$	Backprop through full graph	One back-substitution per θ_j
Hessian $\partial^2\mathbf{x}/\partial\theta_j\partial\theta_k$	Backprop through the Jacobian graph	One back-substitution per (θ_j, θ_k)

The Hessian is where implicit differentiation pays off most: the Kennett method requires AD to differentiate *through* the already expensive Jacobian computation, doubling the graph depth, while the Global Matrix Method reuses the same LU factorisation for every derivative order. Benchmarks show $\sim$1.7$\times$ speedup for the Hessian, compared to $\sim$1.2$\times$ for the Jacobian (see `GlobalMatrix/investigate_ad.py`).

Remark 3. In PyTorch, `torch.linalg.solve` implements implicit differentiation in its backward pass automatically. `torch.func.hessian` applies the identity (66) twice through the autograd graph without the user writing any derivative formulae.

11.4 Block-tridiagonal structure and the Riccati connection

The global matrix $\mathbf{G}$ is *block-tridiagonal*: the displacement-stress continuity at interface i couples only the wave coefficients of layers i and $i{+}1$. Partitioning $\mathbf{G}$ into 4×4 blocks (P-SV up/down amplitudes per layer):

$$\mathbf{G} = \begin{pmatrix} \mathbf{A}_0 & \mathbf{C}_0 & & & \\ \mathbf{B}_1 & \mathbf{A}_1 & \mathbf{C}_1 & & \\ & \mathbf{B}_2 & \mathbf{A}_2 & \ddots & \\ & & \ddots & \ddots & \mathbf{C}_{N-2} \\ & & & \mathbf{B}_{N-1} & \mathbf{A}_{N-1} \end{pmatrix}, \tag{68}$$

where $\mathbf{A}_i$, $\mathbf{B}_i$, $\mathbf{C}_i$ are 4×4 blocks assembled from the displacement-stress eigenvector matrices $\mathbf{E}_i$ and phase factors of adjacent layers.

The optimal direct solver for this structure is the **block-Thomas algorithm** (block-LU without fill-in), which sweeps upward through the interfaces eliminating one block row at a time. At each step, the algorithm computes a **Schur complement**:

$$\mathbf{S}_i = \mathbf{A}_i - \mathbf{B}_i\, \mathbf{S}_{i+1}^{-1}\, \mathbf{C}_i, \tag{69}$$

with $\mathbf{S}_{N-1} = \mathbf{A}_{N-1}$. This is a matrix Riccati recursion — and it is *algebraically identical* to the Kennett addition formula (2).

Proposition 2 (Kennett $\equiv$ Riccati block solver)**.** *The Kennett recursive reflectivity method is the Riccati block solver applied to the block-tridiagonal global matrix system. The cumulative reflection matrix* $\mathbf{RR}_d^{(i)}$ *is the Schur complement of the sub-system below interface* i*, expressed in wave-amplitude coordinates rather than displacement-stress coordinates. The two formulations are related by the eigenvector transformation:*

$$\mathbf{RR}_d^{(i)} = \mathbf{P}\, \mathbf{S}_i^{-1}\, \mathbf{Q}, \tag{70}$$

where $\mathbf{P}$ *and* $\mathbf{Q}$ *are projections from the displacement-stress basis to the up/downgoing wave-amplitude basis.*

This identity has an important consequence: the 1991 thesis (Nestor) used a Riccati block solver for the forward problem, which was *already optimal* — $O(N)$ in the number of layers, with each step involving only 4×4 block operations. The current implementation uses a dense $O(N^3)$ solver (`torch.linalg.solve`), which is a step backwards for the forward solve but gains the implicit differentiation advantage for derivatives.

Unifying perspective. The three approaches form a hierarchy:

Method	Forward solve	Derivatives
Kennett recursion	$O(N)$ block-Riccati sweep	AD through recursive graph
GMM + Riccati solver (1991 thesis)	$O(N)$ block-Riccati sweep	Hand-derived (intractable for Hessian)
GMM + dense solver	$O(N^3)$ dense LU	Implicit diff via `torch.linalg.solve`
GMM + Riccati solver (current)	$O(N)$ differentiable block-Riccati	Implicit diff through each 4×4 block solve

This is precisely what the Riccati solver described in the next subsection implements: the $O(N)$ forward complexity of the block-Riccati sweep combined with PyTorch's implicit differentiation through each 4×4 `torch.linalg.solve` call.

11.5 Differentiable Block-Riccati Solver

The Block-Riccati sweep reduces the global matrix solve from $O(N^3)$ to $O(N)$ per frequency while preserving differentiability via `torch.linalg.solve`. The key insight is that the reflectivity can be computed by propagating a 2×2 Riccati matrix through the layer stack, where each propagation step requires solving a single 4×4 linear system. Since `torch.linalg.solve` provides implicit differentiation in its backward pass, the Jacobian and Hessian are computed automatically through the chain of block solves.

Riccati variable. Define the 2×2 matrix $\mathbf{Y}_k$ as the "effective reflection" at the bottom of finite elastic layer k: it maps the downgoing amplitudes $\mathbf{D}_k$ (referenced at the top of layer k) to the upgoing amplitudes $\mathbf{U}_k$ (referenced at the bottom):

$$\mathbf{U}_k = \mathbf{Y}_k \, \mathbf{D}_k. \tag{71}$$

The Riccati matrix encodes all reflections from the layer stack below interface k.

Upward sweep (bottom → top). Initialise: $\mathbf{Y} = \mathbf{0}$ (radiation condition at the half-space — no upgoing waves from below).

For $k = M, M-1, \ldots, 1$ (where M = number of finite elastic layers), the Riccati matrix is updated by solving a 4×4 linear system at each interface:

1. **Effective downgoing matrix** from the layer below $(k+1)$:

$$\underset{4\times 2}{\mathbf{E}_{\text{eff}}} = \mathbf{E}_d^{(k+1)} + \mathbf{E}_u^{(k+1)} \, \mathrm{diag}(\mathbf{p}_{k+1}) \, \mathbf{Y}, \tag{72}$$

   where $\mathbf{p}_{k+1} = (e^{i\omega\eta_{k+1}h_{k+1}},\ e^{i\omega\nu_{k+1}h_{k+1}})$ is the phase diagonal. For the half-space ($k+1 = N$): $\mathbf{E}_{\text{eff}} = \mathbf{E}_d^{(N)}$.

2. **Phased downgoing** of layer k:

$$\underset{4\times 2}{\mathbf{A}_d} = \mathbf{E}_d^{(k)}\ \mathrm{diag}(\mathbf{p}_k). \tag{73}$$

3. **Assemble and solve** the 4×4 system:

$$\boxed{\underbrace{\left[\mathbf{E}_u^{(k)} \mid -\mathbf{E}_{\mathrm{eff}}\right]}_{4\times 4} \underbrace{\begin{pmatrix} \mathbf{Y}_{\mathrm{new}} \\ \mathbf{Z} \end{pmatrix}}_{4\times 2} = -\mathbf{A}_d.} \tag{74}$$

4. **Extract:** $\mathbf{Y} \leftarrow \mathbf{Y}_{\mathrm{new}}$ (top two rows of the solution).

The physical meaning of (74) is the continuity condition at the bottom of layer k:

$$\mathbf{E}_d^{(k)}\ \mathrm{diag}(\mathbf{p}_k)\,\mathbf{D}_k + \mathbf{E}_u^{(k)}\,\mathbf{U}_k = \mathbf{E}_{\mathrm{eff}}\,\mathbf{D}_{k+1}, \tag{75}$$

with the Riccati substitution $\mathbf{U}_k = \mathbf{Y}_{\mathrm{new}}\,\mathbf{D}_k$ and $\mathbf{D}_{k+1} = \mathbf{Z}\,\mathbf{D}_k$.

Ocean-bottom extraction. After the sweep, $\mathbf{Y} = \mathbf{Y}_1$ (the Riccati matrix at the top of layer 1). The effective eigenvector matrix at the ocean bottom is:

$$\mathbf{E}_1^{\mathrm{eff}} = \mathbf{E}_d^{(1)} + \mathbf{E}_u^{(1)}\ \mathrm{diag}(\mathbf{p}_1)\,\mathbf{Y}_1. \tag{76}$$

The ocean-bottom continuity conditions (u_z, σ_{zz}, and $\sigma_{xz} = 0$) yield a 2×2 system for $\mathbf{D}_1$ after eliminating the ocean upgoing amplitude U_0^P:

$$\begin{pmatrix} e_u^{(0)}[0]\ \mathbf{E}_1^{\mathrm{eff}}[2,:] - e_u^{(0)}[1]\ \mathbf{E}_1^{\mathrm{eff}}[1,:] \\ \mathbf{E}_1^{\mathrm{eff}}[3,:] \end{pmatrix} \mathbf{D}_1 = \begin{pmatrix} \left(e_u^{(0)}[0]\, e_d^{(0)}[1] - e_u^{(0)}[1]\, e_d^{(0)}[0]\right) e_0 \\ 0 \end{pmatrix}, \tag{77}$$

where $e_0 = \exp(i\omega\eta_0 h_0)$ is the one-way ocean phase. The upgoing P-wave amplitude is recovered from u_z continuity:

$$U_0^P = \frac{\mathbf{E}_1^{\mathrm{eff}}[1,:] \cdot \mathbf{D}_1 - e_d^{(0)}[0]\, e_0}{e_u^{(0)}[0]}, \tag{78}$$

and the reflectivity is $R = e_0\, U_0^P$.

Implicit differentiation through the sweep. Each step of the Riccati sweep calls `torch.linalg.solve` on a 4×4 system (74), and the ocean extraction calls it on a 2×2 system (77). PyTorch's autograd provides implicit differentiation through each solve automatically:

For a system $\mathbf{F}\mathbf{X} = \mathbf{C}$, the adjoint (backward pass) computes

$$\bar{\mathbf{C}} = \mathbf{F}^{-T}\bar{\mathbf{X}}, \qquad \bar{\mathbf{F}} = -\bar{\mathbf{C}}\,\mathbf{X}^T, \tag{79}$$

where $\bar{(\cdot)}$ denotes the adjoint (gradient of the loss with respect to that quantity). The chain rule through the Riccati sweep composes these adjoint operations backward through the interfaces, yielding the Jacobian $\partial R/\partial\theta_j$ at cost $O(N \times 4^3)$ per frequency — identical complexity to the forward sweep.

Hessian. `torch.func.hessian` computes the second-order derivatives by applying reverse-mode AD twice: once to form the gradient of the scalar misfit, then a second reverse pass to differentiate the gradient. Each level of differentiation passes through the same chain of block solves, using the implicit differentiation identity at each step. The total Hessian cost is $O(N \times P^2)$ per frequency, where P is the number of model parameters — the same as the dense solver, but with the forward pass reduced from $O(N^3)$ to $O(N)$.

Complexity summary.

Quantity	Dense GMM	Riccati GMM
Forward $R(\omega)$	$O(N_\omega \cdot N^3)$	$O(N_\omega \cdot N \cdot 4^3)$
Jacobian $\partial R/\partial\theta_j$	$O(N_\omega \cdot N^3 + N_\omega \cdot P \cdot N^2)$	$O(N_\omega \cdot N \cdot P \cdot 4^3)$
Hessian $\partial^2\chi/\partial\theta_j\partial\theta_k$	$O(N_\omega \cdot P^2 \cdot N^2)$	$O(N_\omega \cdot P^2 \cdot N \cdot 4^3)$

For $N = 100$ layers, the Riccati solver is approximately 60,000× faster for the forward solve. The derivative advantage is smaller (the dense solver reuses the LU factorisation) but still significant: $O(N)$ versus $O(N^2)$ per Jacobian column.

Edge cases.

- **2-layer model** ($M = 0$): The Riccati sweep is empty. The ocean couples directly to the half-space via $\mathbf{E}_1^{\text{eff}} = \mathbf{E}_d^{(1)}$.
- **3-layer model** ($M = 1$): A single Riccati step, then ocean extraction.
- **Free surface**: Applied identically after extracting R: $R_{\text{fs}} = R/(1 + R)$.

12 Levenberg–Marquardt Inversion

12.1 Misfit functional

Given observed plane-wave reflectivity $R_{\text{obs}}(\omega_i, p_s)$ at frequencies ω_i ($i = 1, \ldots, N_\omega$) and ray parameters p_s ($s = 1, \ldots, N_p$), the L2 misfit is:

$$\chi(\boldsymbol{\theta}) = \sum_{s=1}^{N_p} \sum_{i=1}^{N_\omega} \left| R(\omega_i, p_s; \boldsymbol{\theta}) - R_{\text{obs}}(\omega_i, p_s) \right|^2. \tag{80}$$

12.2 Gradient

The gradient of the misfit is:

$$g_k = \frac{\partial \chi}{\partial \theta_k} = 2\,\mathrm{Re} \sum_{s,i} \overline{r_{si}}\, \frac{\partial R_{si}}{\partial \theta_k}, \tag{81}$$

where $r_{si} = R(\omega_i, p_s; \boldsymbol{\theta}) - R_{\text{obs}}(\omega_i, p_s)$ is the complex residual. In matrix form:

$$\mathbf{g} = 2\,\mathrm{Re}\big(\mathbf{J}^H \mathbf{r}\big), \tag{82}$$

where $\mathbf{J}$ is the Jacobian matrix with entries $J_{(s,i),k} = \partial R_{si}/\partial\theta_k$ and $\mathbf{r}$ is the residual vector.

12.3 Hessian decomposition

The Hessian of the misfit decomposes as (cf. Section 10):

$$H_{kl} = \frac{\partial^2 \chi}{\partial \theta_k\, \partial \theta_l} = \underbrace{2\,\mathrm{Re} \sum_{s,i} \overline{\frac{\partial R_{si}}{\partial \theta_k}}\, \frac{\partial R_{si}}{\partial \theta_l}}_{\text{Gauss–Newton: } \mathbf{H}_{\text{GN}} = 2\,\mathrm{Re}(\mathbf{J}^H \mathbf{J})} + \underbrace{2\,\mathrm{Re} \sum_{s,i} \overline{r_{si}}\, \frac{\partial^2 R_{si}}{\partial \theta_k\, \partial \theta_l}}_{\text{second-order residual term}}. \tag{83}$$

The Gauss–Newton approximation $\mathbf{H} \approx \mathbf{H}_{\text{GN}}$ drops the second-order term. This is valid near the solution where $\|\mathbf{r}\| \to 0$, but fails far from the solution.

12.4 The Levenberg–Marquardt update

At each iteration, the parameter update $\delta\boldsymbol{\theta}$ solves:

$$\boxed{\left(\mathbf{H} + \lambda\,\mathbf{I}\right)\delta\boldsymbol{\theta} = -\mathbf{g},} \tag{84}$$

where $\mathbf{H}$ is the exact Hessian (83), $\mathbf{g}$ is the gradient (82), and $\lambda > 0$ is the damping parameter.

Damping strategy:

- If $\chi(\boldsymbol{\theta} + \delta\boldsymbol{\theta}) < \chi(\boldsymbol{\theta})$: accept the step, decrease $\lambda \leftarrow \lambda/\nu$ (trust the Newton direction).
- If $\chi(\boldsymbol{\theta} + \delta\boldsymbol{\theta}) \geq \chi(\boldsymbol{\theta})$: reject the step, increase $\lambda \leftarrow \nu\lambda$ (more regularisation).

For $\lambda \to 0$ the update approaches the full Newton step with quadratic convergence; for $\lambda \to \infty$ it approaches the steepest-descent direction $\delta\boldsymbol{\theta} \propto -\mathbf{g}$.

12.5 Log-parameter transformation

To enforce positivity of physical parameters and improve conditioning, the inversion operates in log-parameter space:

$$\boldsymbol{\phi} = \log\boldsymbol{\theta} = \big(\log\alpha_1, \ldots, \log\alpha_N,\ \log\beta_1, \ldots, \log\beta_N,\ \log\rho_1, \ldots, \log\rho_N,\ \log h_1, \ldots, \log h_{N-1}\big). \tag{85}$$

The Jacobian in log-space is related to the physical-space Jacobian by:

$$\frac{\partial R}{\partial \phi_k} = \frac{\partial R}{\partial \theta_k}\,\theta_k = \theta_k\,\frac{\partial R}{\partial \theta_k}, \tag{86}$$

which is equivalent to a diagonal scaling $\mathbf{J}_\phi = \mathbf{J}\ \mathrm{diag}(\boldsymbol{\theta})$. The Hessian transforms correspondingly:

$$\frac{\partial^2\chi}{\partial\phi_k\,\partial\phi_l} = \theta_k\,\theta_l\,\frac{\partial^2\chi}{\partial\theta_k\,\partial\theta_l} + \delta_{kl}\,\theta_k\,\frac{\partial\chi}{\partial\theta_k}, \tag{87}$$

where δ_{kl} is the Kronecker delta. Both transformations are handled automatically by PyTorch AD when the forward model takes $\exp(\boldsymbol{\phi})$ as input.

12.6 Convergence properties

Method	Hessian used	Convergence rate
Steepest descent	none	Linear: $\lVert\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\rVert \leq c\,\lVert\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\rVert$
Gauss–Newton	$\mathbf{J}^H\mathbf{J}$	Superlinear (near solution)
Full Newton (LM)	exact $\mathbf{H}$	Quadratic: $\lVert\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\rVert \leq c\,\lVert\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\rVert^2$

The 1991 thesis used the Gauss–Newton approximation because hand-deriving the full Hessian through the Kennett recursion was intractable. With automatic differentiation, the exact Hessian is obtained at no additional implementation effort — only computational cost — enabling true quadratic convergence of the Levenberg–Marquardt iteration.

References

[1] B.L.N. Kennett, *Seismic Wave Propagation in Stratified Media*, Cambridge University Press, 1983.

[2] M. Dietrich and F. Kormendi, "Perturbation of the plane-wave reflectivity of a depth-dependent elastic medium by weak inhomogeneities," *Geophys. J. Int.*, vol. 100, pp. 203–214, 1990.

[3] F. Kormendi and M. Dietrich, "Nonlinear waveform inversion of plane-wave seismograms in stratified elastic media," *Geophysics*, vol. 56, pp. 664–674, 1991.

[4] R.C.Y. Chin, G.W. Hedstrom and L. Thigpen, "Matrix methods in synthetic seismograms," *Geophys. J. R. astr. Soc.*, vol. 77, pp. 483–502, 1984.

[5] D.W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *SIAM J. Appl. Math.*, vol. 11, pp. 431–441, 1963.

[6] T.M. Nestor, *A Practical Implementation of Waveform Inversion in a Stratified Marine Environment*, Master of Science thesis, Monash University, 1991.